CampusFlow

Venue Seating, Check-in and Crowd Evacuation Planner
built on five hand-written data structures

Team: The CodeSmiths

Srivathsav	106125061
Manmeet	106125063
Manthan	106125065

CSLR31 – Data Structures Laboratory Project Report

Step 1: Abstract

Problem and motivation. Campus events such as seminars, examinations and cultural programmes can bring hundreds of people into a venue. When seating is managed manually, it becomes hard to keep track of available seats, accommodate late arrivals fairly and know how many people are present in each part of the hall. Seats may also remain empty when registered participants do not show up. In an emergency, a blocked exit or a crowded corridor can make it difficult to guide people along a safe alternative route.

Existing products. Online ticketing platforms allow users to choose seats, while crowd-monitoring systems use cameras or sensors to estimate how many people are present. These solutions usually handle separate parts of the problem and may require paid services or additional hardware. CampusFlow brings seating, waiting-list management and evacuation routing together in a single terminal-based program.

Objectives. We developed CampusFlow as a terminal-based system with four main goals. It registers participants and tries to seat each person in their preferred zone; maintains a priority-based waiting list and assigns released seats to waiting participants; reports occupancy and crowding levels for each zone; and finds routes through the venue while taking congestion into account, including during evacuation.

Salient features. The system uses a hash table for participant lookup in $O(1)$ average time and a segment tree to calculate zone totals and locate the first free seat in $O(\log N)$. A min-heap selects the next participant from the waiting list based on priority and arrival time. For movement around the venue, the graph module uses Dijkstra's algorithm, BFS and a multi-source evacuation search. Congestion alerts use hysteresis: they switch on at 80% occupancy and switch off at 70%, which helps prevent alerts from repeatedly turning on and off.

Future scope. There is room to extend the project with a graphical interface, persistence for the waiting list, live head-counts through QR or RFID check-in, and our own implementations of the standard containers that are still used in a few modules.

Social cause and SDG alignment. CampusFlow is related to UNESCO's Sustainable Development Goal 11 (*Sustainable Cities and Communities*) because it aims to make shared spaces, including campus halls, safer and easier to manage. It also connects with Goal 4 (*Quality Education*), as better-organised campus events can help make educational activities more accessible to registered participants [1].

1 Step 2: Design of the Product

1.1 Idea

For this project, we represent the venue as a 6×8 grid containing 48 seats. The grid is divided into three horizontal zones, with 16 seats in each zone. Every participant is assigned a priority (0–9) and selects a preferred zone. We built the design around four main ideas:

1. **Keep occupancy data consistent.** Whenever a seat changes state, the seat grid updates the segment tree as part of the same operation. Reports get their counts from the tree instead of maintaining separate counters, which helps avoid mismatched values.
2. **Handle a full venue fairly.** When no seat is available, participants are added to a min-heap. The heap considers priority first and join time next, so the participant with the best waiting-list rank is selected first.
3. **Reuse seats after no-shows.** A participant gets a 15-minute grace period to check in. If they do not arrive, the system releases the seat and offers it to the next eligible person on the waiting list.
4. **Connect seating with movement.** Before a graph operation is carried out, the occupancy of the three zone halls is refreshed from the segment tree. This allows the routing and evacuation calculations to use the latest occupancy information.

1.2 Flow chart

Figure 1 gives an overview of how the program runs. It starts by initialising the data structures and then displays the main menu. After the user selects an option, the corresponding operation is performed and control returns to the menu. This continues until the user selects option 0 to exit. We created the flow chart using Graphviz and included it in the report as `flowchart.pdf`.

1.3 Modules and how they work

The project is divided into the modules listed in Table 1. For most modules, the header file contains the declarations and the source file contains the corresponding definitions.

Registration and seat allocation

When a participant registers, the system stores their details in the hash table. It rejects the entry if the ID is empty or already exists. During seat allocation, the segment tree is used to find the first available general seat in the participant's preferred zone through `findFirstFree`. If that zone has no free seat, the system checks the nearest zone. When every usable seat is taken, the participant is added to the waiting list. Blocked seats are excluded from the venue's capacity, while reserved seats are skipped during automatic allocation.

Check-in, no-shows and the waiting list

When a participant checks in, their seat changes from *allocated* to *occupied*. Option 16 advances the simulated clock. Once 15 minutes have passed, option 17 releases the seats belonging to participants who have not checked in, marks those participants as NO_SHOW and assigns the freed seats to people from the waiting list. The heap ranks participants by higher priority first, earlier join time second and ID as the final tie-breaker. This also makes the selection order predictable.

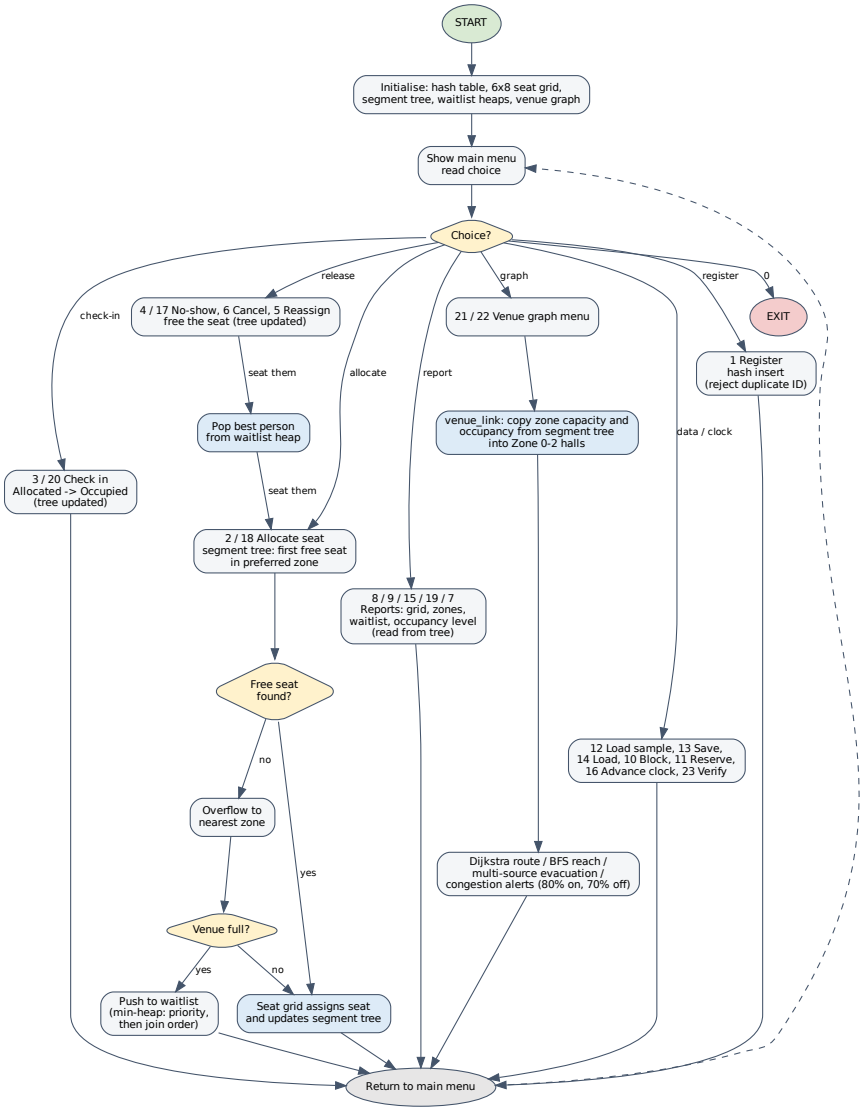


Figure 1: Flow chart of CampusFlow. Shaded boxes mark steps where one data structure hands work to another.

Occupancy report

To generate an occupancy report, the program queries the segment tree for each zone and then for the venue as a whole. A participant is counted as present only after checking in. The program uses the occupancy percentage to assign a level, following the thresholds shown in Table 1.

Venue graph, routing and evacuation

The default venue map contains seven locations: `Gate`, `Junction`, `Zone 0`, `Zone 1`, `Zone 2`, `Exit East` and `Exit West`. Corridors connect these locations. Each corridor is assigned a cost using the following formula:

$$\text{cost} = t_{\text{normal}} \times (1 + p \times r),$$

Here, t_{normal} is the corridor's normal travel time, p is its congestion likelihood (0–1), and r is the occupancy ratio at the busier end of the corridor. Closed locations and corridors are ignored during route calculations.

Module	Files	Responsibility
Hash table	`hash_table.*`	Finds a participant by ID string; chaining; doubles in size when load reaches 0.75.
Seat grid & allocation	`seat_grid.*`, `allocation.*`	Holds the state of every seat (free, allocated, occupied, blocked, reserved); allocates, reassigns, cancels; keeps the grid, hash table and tree in agreement.
Segment tree	`segment_tree.*`	Per-range counts of free, reserved, allocated and occupied seats; point update, range query, find first free seat.
Waitlist	`waitlist_heap.*`, `waitlist.*`	Min-heap per zone; no-show release and automatic promotion with the 15-minute grace period.
Occupancy	`occupancy.*`	Zone and venue report with levels LOW (below 30%), NORMAL (below 70%), HIGH (below 90%), OVERCROWDED.
Venue graph	`VenueGraph.h`, `Config.h`, `graph_menu.*`	Locations and corridors; Dijkstra, BFS, multi-source evacuation, congestion alerts with hysteresis; graph menu and custom venue setup.
Link	`venue_link.*`	Copies zone capacity and occupancy from the segment tree into the graph.
Persistence	`persistence.*`	Saves and loads participants and blocked/reserved seats as plain text; rejects a missing, corrupt or wrong-size file without changing data.
Main menu	`main.cpp`	The 23-option menu that calls all modules.

Table 1: Modules of CampusFlow.

We use *Dijkstra's algorithm* to find the fastest route between two locations, and *BFS* to list all locations reachable from a given point. For evacuation, a *multi-source search* starts from all open exits at the same time. This finds the best exit for every location in one pass, while also considering clearance times. Congestion alerts turn on when occupancy reaches 80% of capacity and turn off when it falls to 70%. Between these thresholds, the alert keeps its previous state; this is the hysteresis behaviour.

Complexity summary

Structure	Operation	Cost	Why chosen
Hash table	insert, find, erase	$O(1)$ average	Direct lookup from an ID string
Seat grid	change one seat	$O(1)$	Natural 2-D layout
Segment tree	update, range count, first free	$O(\log N)$	No scan of all seats
Min-heap	push, pop best	$O(\log W)$	Always serves the best waiting person
Min-heap	cancel from the middle	$O(W)$	Linear search by ID
Graph	Dijkstra / BFS	$O((V+E)\log V)$ / $O(V+E)$	Routes with congestion cost

Table 2: Data structures, operations and costs (N seats, W waiting, V locations, E corridors).

2 Step 3: Coding

2.1 Modular structure

We have kept the code organised into separate files. As required by the assignment, declarations are placed in `.h` files inside `include/`, while definitions are placed in `.cpp` files inside `Source/`. Compiled object files are stored in `object/`, and the `Makefile` and executable `a.out` are kept in the parent directory. The complete project contains approximately 3400 lines of code, including the tests.

Listing 1: Directory layout of the submission.

```
campusflow/
|-- Makefile, a.out
|-- include/     13 headers (.h)
|-- Source/      11 source files (.cpp), including main.cpp
|-- object/      compiled .o files
|-- tests/       5 test programs
|-- samples/     scripted inputs and expected outputs
|-- Document/    user manual
```

The project can be built and tested using the commands below. Running `make` creates the `a.out` executable, and `./a.out` starts the program. The command `make test` runs the test suites, while `make demo` replays the sample sessions.

2.2 Sample code

The code samples below show how we implemented some of the main data structures. In each case, the header file declares the interface and the source file contains the implementation. The first example is the segment tree interface from `include/segment_tree.h`:

Listing 2: Segment tree interface (excerpt).

```
class SegmentTree {
public:
    void build(const std::vector<TreeSeat>& seats);
    void update(int idx, TreeSeat s);       // O(log N)
    Counts query(int l, int r) const;       // inclusive range, O(log N)
    int findFirstFree(int l, int r, FreeKind kind) const;   // -1 if none
private:
    int n = 0;
    std::vector<Counts> tree;               // 4*n nodes, node 1 = root
};
```

The next example is the search function from `Source/segment_tree.cpp`. It finds the first free seat in a requested range in $O(\log N)$ time. To save work, it skips any part of the tree that has no free seats:

Listing 3: Finding the first free seat in a range.

```
int SegmentTree::findRec(int node, int l, int r, int ql, int qr,
                         FreeKind kind) const {
    if (qr < l || r < ql) return -1;                // no overlap
    if (freeOf(tree[node], kind) == 0) return -1;   // nothing free below
    if (l == r) return l;                           // found a leaf
    int m = (l + r) / 2;
    int res = findRec(2 * node, l, m, ql, qr, kind);
    if (res != -1) return res;                      // left side wins
    return findRec(2 * node + 1, m + 1, r, ql, qr, kind);
}
```

The final example shows how the waiting-list heap compares two entries and moves a newly inserted entry towards its correct position (`Source/waitlist_heap.cpp`):

Listing 4: Heap ordering and sift-up.

```
bool WaitlistHeap::better(const WaitEntry& a, const WaitEntry& b) {
    if (a.weight != b.weight) return a.weight > b.weight;   // priority first
    if (a.regTime != b.regTime) return a.regTime < b.regTime; // then earlier
    return a.id < b.id;                                      // then ID
}

void WaitlistHeap::siftUp(int i) {
    while (i > 0) {
        int p = (i - 1) / 2;
        if (!better(h[i], h[p])) break;
        std::swap(h[i], h[p]);
        i = p;
    }
}
```

2.3 Honest note on standard containers

We implemented the hash table, seat grid, segment tree, heap and graph algorithms ourselves. However, a few parts of the program still rely on C++ standard containers. The hash table uses `std::list` for its buckets, and the waiting-list module uses `std::unordered_set` to track waiting IDs. The graph module uses `std::unordered_map` for name lookup, `std::priority_queue` in Dijkstra's algorithm and `std::queue` in BFS. Replacing these remaining containers with our own implementations is one of the improvements we have left for future work.

3 Step 4: Testing

3.1 Automated test suites

We used five automated test suites to check the different parts of the system. The command `make test` builds and runs all five suites. A suite returns exit code 0 only when all of its checks pass. In our final build, all five suites completed successfully.

Suite	What it covers
`test_all`	Hash table (collisions, resize, duplicates), seat grid, allocation, zone overflow, tree/grid agreement, save/load, edge cases.
`test_waitlist`	Priority and zone order, no-show release, empty waiting list, a randomised run with 1604 people.
`test_occupancy`	Level thresholds, per-zone levels, edge cases, last-seat case, comparison with a brute-force count.
`test_graph` (117 checks)	Graph validation, Dijkstra (disconnected, closed location, closed corridor, congestion changing the route), BFS, multi-source evacuation (several exits, closed exit, all exits closed), hysteresis, capacity update.
`test_integration` (178 checks)	Whole system: registration to check-in feeding the graph; alerts driven by cancellations; waiting list and no-show flow; evacuation reroute; save/load round trip; 3000-step random stress test comparing the segment tree with brute-force counts.

Table 3: Automated test suites.

3.2 Test data

- Venue: 6×8 grid, 48 seats, 3 zones of 16 seats; no-show grace period 15 minutes.
- Option 12 registers participants `P101, P102, ...` with rotating priority 0–2 and rotating preferred zone; the demo uses 50, plus one hand-entered participant `A100` (priority 5, zone 1), so 51 people in total.
- Seat (5,7) is blocked and seat (5,6) is reserved, leaving 46 usable seats.
- Three scripted sessions in `samples/`: `full_demo`, `custom_venue` and `edge_cases`.

3.3 Functional test cases

The test cases in Table 4 are taken from the scripted `full_demo` session. When we run `make demo`, the program replays the session and checks its output against the saved expected output. All three sample sessions match their expected results. We also checked that the program exits cleanly if the input ends before the user completes an operation.

Table 4: Functional test cases (from `samples/full_demo_input.txt`).

No.	Function	Input	Expected result (observed: pass)
1	Register	Option 1, `A100`	Participant registered
2	Duplicate ID	Option 1, `A100` again	Rejected: ID already exists
3	Unknown participant	Option 7, `NOPE`	Reports not found
4	Load sample data	Option 12, 50	50 participants `P101`–`P150`
5	Block / reserve seat	Option 10 (5,7); option 11 (5,6)	Seat blocked; seat reserved; 46 seats usable
6	Full venue	Option 18	46 seated, 5 waitlisted
7	Waiting list order	Option 15	Per zone, higher priority first, then earlier join
8	Consistency	Option 23	"Consistent: hash table, seat grid and segment tree agree"
9	Check-in a zone	Option 20, zone 0	16 checked in; Zone 0 is 100.0% and OVERCROWDED
10	Evacuation	Option 21, then 8, `Zone 0`	Route Zone 0 → Exit West, cost 46.00 s
11	Fastest route	Graph option 6, `Gate` to `Exit East`	Gate → Junction → Zone 2 → Exit East, 52.00 s
12	Closed exit reroute	Close `Exit West`, then option 8, `Zone 0`	Route changes to Zone 0 → Zone 1 → Zone 2 → Exit East, 96.00 s
13	Early no-show check	Advance clock 10, then option 17	Refused: grace period not over
14	No-show release	Advance clock 5 more, then option 17	14 no-show seats released; 5 waiting people seated in priority order
15	Reassign seat	Option 5, `P148` to (4,0)	Reassigned; grid shows the move
16	Cancel	Option 6, `P101`	Cancelled; participant shows CANCELLED
17	Save and load	Options 13 and 14	State restored; still consistent

3.4 Edge cases

The `edge_cases` session covers situations that could cause problems in normal use. These include invalid numbers, unknown IDs, duplicate registrations, invalid seat positions, checking for no-shows too early, files that are missing or cannot be written, all exits being closed, and crowd values outside

the allowed range. If a save file is missing, corrupt or has the wrong size, the program rejects it without changing the current data.

3.5 A defect found by testing

We found one defect while combining the work from all three team members. When a seat was reassigned to someone who had already checked in, the participant's record still showed them as checked in, but the new seat was marked only as *allocated*. As a result, the occupancy report did not count that participant as present. We fixed the issue with a one-line change in `allocation.cpp` and added an integration test so that the same problem is caught if it appears again.

4 Step 5: Contributions, Key Takeaways and References

4.1 Individual contributions

The work was divided across the three team members so that each person could focus on a set of related modules. Table 5 records the files handled by each member and summarises their main contributions.

Member	Files owned	Main contribution
Srivathsav (106125061)	`common.h`, `hash_table.*`, `seat_grid.*`, `allocation.*`, `persistence.*`, `test_all.cpp`	Hash table, seat grid, seat allocation, zone overflow, reassign and cancel, save/load, and their tests.
Manmeet (106125063)	`segment_tree.*`, `waitlist_heap.*`, `waitlist.*`, `occupancy.*`, `test_waitlist.cpp`, `test_occupancy.cpp`	Segment tree, waiting-list heap, check-in and no-show handling, occupancy report, and their tests.
Manthan (106125065)	`Config.h`, `VenueGraph.h`, `graph_menu.*`, `test_graph.cpp`	Venue graph, Dijkstra, BFS, evacuation, congestion alerts, analytics, crowd simulation, graph menu, and tests.
Shared	`main.cpp`, `venue_link.*`, `test_integration.cpp`, `Makefile`, `samples/`	Main menu, link from the tree to the graph, integration tests, build file and sample sessions.

Table 5: Files owned by each member.

4.2 Key takeaways

- We learned how useful a **segment tree** can be for handling both zone-wise counts and first-free-seat queries in $O(\log N)$. Using it as the common source of occupancy data also helped us avoid inconsistent counts.
- A **min-heap** can do more than store values by size. By comparing priority, join time and ID, we were able to make waiting-list selection fair and predictable.
- The **hash table** and its resizing logic helped us keep participant lookup close to $O(1)$ on average as the number of registrations increased.
- Using **Dijkstra, BFS and multi-source search** on the same venue graph showed us how different graph algorithms can solve different movement-related problems. Including congestion in the edge cost can also change the route that is selected.

- Working with a **modular codebase**, a Makefile and separate tests made it easier for three people to develop different parts in parallel. The integration tests were especially useful because they exposed a defect that had not been caught earlier.

4.3 Future scope

- Include the waiting list in saved data. At present, it has to be rebuilt using option 18 after a file is loaded.
- Replace the standard containers that are still used in a few modules with our own implementations.
- Develop a graphical or web-based interface and support live head-counts through QR or RFID check-in.
- Add an option to switch from a custom venue graph back to the default graph without restarting the program.

4.4 Acknowledgements

We are grateful to our course instructor for providing the project problem statement and guiding us throughout CSLR31. We also thank the laboratory staff for their support and acknowledge the effort and cooperation of all our team members in completing the project.

References

[1] UNESCO, *Sustainable Development Goals*. `https://www.unesco.org/en/sdgs`

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, MIT Press (heap, hash table, graph search and Dijkstra's algorithm).

[3] E. W. Dijkstra, "A note on two problems in connexion with graphs", *Numerische Mathematik*, 1:269–271, 1959.

[4] cppreference.com, *C++ reference* (standard library containers used).

[5] GNU Make Manual, `https://www.gnu.org/software/make/manual/`